FACULTY OF ENGINEERING AND THE BUILT ENVIRONMENT

ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING DEPARTMENT SOFTWARE DESIGN (SDN260S)

Final Submission

Assignment – Log Analyzer

GROUP 36

No.	NAME & SURNAME	STUDENT NUMBER
1.	Bernicio Reece Coetzee	240161009
2.	Jamaine Leeuw	240953819
3.	Hundzukani Milton Msapa	240096495
4.	Iviwe Nomzaza	230452795
5.	Ruramai Molly Rutumhu	230388353
6.	Somila Mancoba	240660102
7.	Reginald McCloen	231173652
8.	Hercy Tchitembo	222571705

Supervisor: Dr Ekwe

Due Date: 08 November 2026

Campus: Bellville

Contents

Introduction and scope 4
Requirements 4
Purpose and users 4
Assumptions and constraints 5
Systems Overview and Architecture 8
Overview 8
Architecture style 8
Layers and components 9
Runtime flow 9
Key design points 9
Detailed design and diagrams 9
Entry point, UI and service layer 10
File access and line index 11
Parsing log lines 12
Detection 12
Findings storage and reporting 13
Key Design Decisions and Trade-Offs 13
Testing and validation 14
Test cases and results (current build) 14
Earlier test runs (before severity levels were added) 16
User guide (README) 21
Overview 21
Requirements and setup 21
Running the application (Apache NetBeans) 21
Command-line interface 21
Graphical interface 22
Graduate attribute reflections 22
GA2: Application of scientific and engineering knowledge 22
GA7: Sustainability and impact 22
Challenges faced 23
Technical 23
Teamwork 23

Team contribution breakdown 23
Limitations and future improvements 24
Limitations 24
Future improvements 24
Conclusion 25
References 25

Introduction and scope

1. **Background and purpose**
   Applications and servers record login activity in log files. Reading these by hand is slow and error-prone, and repeated failed logins from one source are a common sign of a brute-force attack. The Log-Analyzer is a Java application that reads log files, converts their lines into structured events, applies a detection rule, and reports suspicious login behaviour as prioritised findings.
2. **Problem statement**
   Administrators and students reviewing logs need a quick way to find suspicious login attempts without searching thousands of lines manually. Logs also come in different formats, so a useful tool must read several of them and present results clearly. The cost of missing an attack can be high: in July 2021, a cyber attack on Transnet, which operates South Africa's major ports, disrupted container terminal operations at ports such as Durban and Cape Town (Shabalala and Heiberg, 2021), and the company declared force majeure at its container terminals (Heiberg, 2021).
3. **Aim and objectives**
   Aim: to design and build a tool that detects suspicious login attempts in log files and reports them clearly.

Scope

- **Input:** Local log files placed in logs/ , selected by file name (path traversal is rejected).
- **Format:** Java-style text, Laravel, JSON Lines, and a plain-text fallback
- **Detection:** One rule – repeated failed logins from the same IP within a sliding time window
- **Output:** Findings with severity, timestamp, rule, source file and description; analysis summaries and CSV or text exports.
- **Interfaces:** A command-line application and a Swing GUI with a file library, search, severity filters, parsed-entry and malformed-line views.
- **Processing:** Offline analysis, recursive discovery of nested files and a batch mode with up to four worker threads. File contents are cached in memory.

Requirements

Purpose and users

The system reads authentication-related log files detects patterns that suggest suspicious login attempts, such as brute-force guessing, then presents the findings as a prioritised report.

- Primary user: a student or system administrator reviewing logs after an incident or a routine check.

- **Secondary user:** an assessor or developer who runs the tool on a sample to verify behaviour.

Assumptions and constraints

- Logs are text files, one event per line. Supported entries have a valid timestamp, recognised log level and message; JSON requires timestamp and message strings, Analysis is offline (batch), not living monitoring.
- Analysis is offline (batch), not live monitoring.
- Implementation: Java 17, Maven and Swing. NetBeans or IntelliJ can be used for development. The built JAR only requires Java 17 or newer to run.
- Test data is synthetic: The supplied logs cover repeated failed logins, clean activity, malformed lines, JSON, Laravel and 1PV4 boundaries.

Log ingestion

ID	Requirement	Priority	Acceptance criterion	Status
FR1	The system shall let the user select a log file (GUI dropdown, CLI argument).	Must	Selecting sample.log loads it without error	Built (GUI)
FR2	The system shall provide a refresh action to reload the available log files.	Should	New file appears after Refresh	Built (GUI)
FR3	The system shall read large files line by line without loading the entire file into memory.	Should	A 100 MB file is processed without out-of-memory errors	Not met: the whole file is read and cached (D6)
FR4	The system shall report a clear message if the file is missing, empty or unreadable.	Must	No crash; user-readable error shown	Partly built: a non-log file (.gitkeep) runs without a crash

Parsing

ID	Requirements	Priority	Acceptance criterion	Status

FR5	The system shall convert each valid line into a LogEvent containing timestamp, message and extracted fields.	Must	Known line yields expected field values (unit test)	Built (parser/LogEvent)
FR6	The system shall extract the source IP address and username from each login event	Must	Event with IP 203.0.113.42 and user alice parses correctly	Built (regex in rule); the IP pattern is loose (see Limitations)
FR7	The system shall handle malformed lines by skipping and counting them rather than stopping.	Must	File with 3 bad lines still analyses; skip count reported	Not met: the summary shows a malformed count, but it always shows 0
FR8	The system shall normalise timestamps to a single time zone for comparison.	Should	Report shows zone (SAST) consistently	Built
FR9	The system shall keep the original line number/content for each event so findings can be traced back.	Should	Finding can reference its source line (IndexedLine)	Not met: findings do not yet reference line numbers

Detection of suspicious activity

ID	Requirements	Priority	Acceptance	Status
FR10	The system shall detect repeated failed logins: N or more failures from the same IP within a time window T.	Must	5 failures in 5 minutes from one IP produces a HIGH finding (earlier failures produce LOW and MEDIUM findings)	Built

FR11	N and T shall be configurable, with N ≥ 2 and T positive and non-null; invalid values shall be rejected.	Must	N = 1 or T = 0 throws a clear error	Built
FR12	Detection shall use a sliding time window, not fixed clock intervals.	Should	Failures straddling a minute boundary are still counted	Built (sliding window, D7)
FR13	Detection rules shall be independent modules implementing a common DetectionRule interface so new rules can be added without changing the engine.	Must	Adding a rule requires no edits to DetectionEngine logic	Built
FR14	The engine shall run all registered rules over the event stream and collect their findings.	Must	Two rules produce findings from one run	Built (one rule implemented)
FR15	The system shall not report the same incident more than once (de-duplication by rule + IP + window).	Must	Same burst yields exactly one finding	Not met: duplicates appear when Analyze is repeated

Reporting and interface

ID	Requirement	Priority	Acceptance	Status
FR19	The system shall produce a text report	Must	Report shows “Generated findings: N”	Built

	with a header, total finding count and each finding.		matching the list	
FR20	The GUI shall provide Analyze and Report actions and display results in a scrollable area.	Must	Buttons trigger analysis and display	Built
FR21	The CLI shall run the same analysis and print or save the report.	Should	CliApp prints identical findings	Built (CliApp)
FR22	The report shall be exportable to a file (text or CSV).	Could	Saved file opens and matches the on-screen report	Planned
FR23	The report shall show a summary (events read, lines skipped, findings by severity).	Could	Counts match test data	Partly built: the summary shows parsed lines, malformed lines and the findings count

Systems Overview and Architecture

Overview

The Log-Analyzer reads authentication logs, applies detection rules, and reports suspicious login behaviour as prioritised findings

- Input: Local . log, .txt and .jsonl files in logs/ and its subfolders.
- Output: Analysis totals, prioritised findings, parsed and malformed entries, saved history and CSV/text reports
- Interfaces: desktop GUI and a command-line app (CliApp)

Architecture style

A layered, pipe-and-filter design with plug-in detection rules (Strategy pattern). Data flows one way through the stages, which keeps each stage simple and testable, and new rules can be added without changing the engine.

Alternatives rejected: a single monolithic class (untestable), event-driven streaming design (only needed for live monitoring), client-server (unnecessary for local files).

Layers and components

Layer	Package	Key Classes	Role
Presentation	Ui.cli, GUI	CliApp, GuiApp, LogController	Take input, show the report
Service	service	LogViewerService, ReportService, AnalysisResult	Coordinate the analysis
Detection	Detection.rules	DetectionEngine, DetectionRule, Finding, RepeatedLoginFailureRule	Run rules, produce findings
Parsing	parser	ParserRegistry, LogParser implementations (including JsonLinesParser and the PlainTextParser fallback), LogEvent	Turn raw lines into events
Files access	File.index	SafeFileResolver, IndexedLogReader, IndexedLine, MemoryIndexRepository	Read lines with line numbers

Runtime flow

Log file → SafeFileResolver → IndexedLogReader → numbered IndexedLine records → ParserRegistry → valid LogEvents and malformed-line records → DetectionEngine → unique saved Findings → AnalysisResult → GUI / CLI / export

Key design points

- The engine uses DetectionRule so new rules can be registered. Invalid thresholds and timie windows are rejected. Regexes are precompiled and per-IP queues avoid repeated scans. Malformed lines retain their original line numbers while valid events continue.

Detailed design and diagrams

This design separates user interaction, file access, parsing, detection and reporting. The diagram below shows the final analysis flow; the following sections explain the main classes and the advanced features.

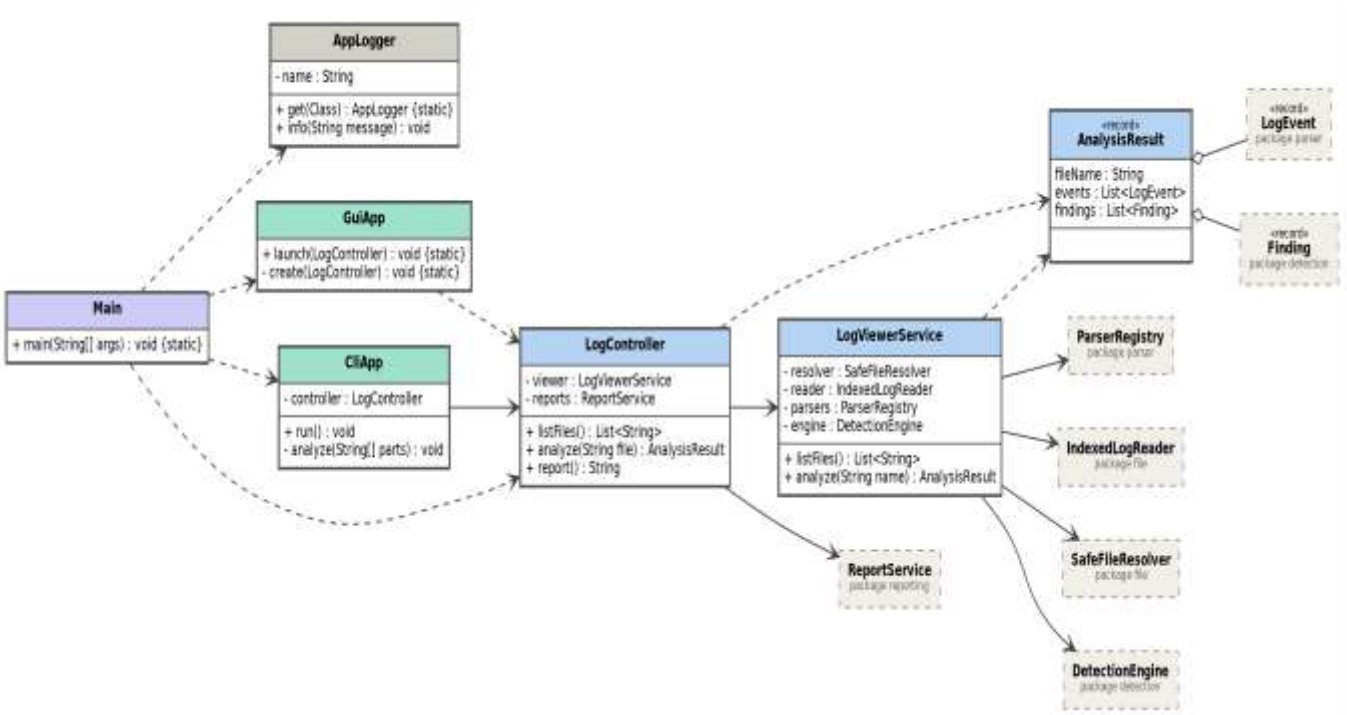


How to read the diagrams

Arrows show the order of processing for one file. LogViewerService coordinates the stages. Analyze all runs up to four file pipelines concurrently; SwingWorker keeps the desktop interface responsive.

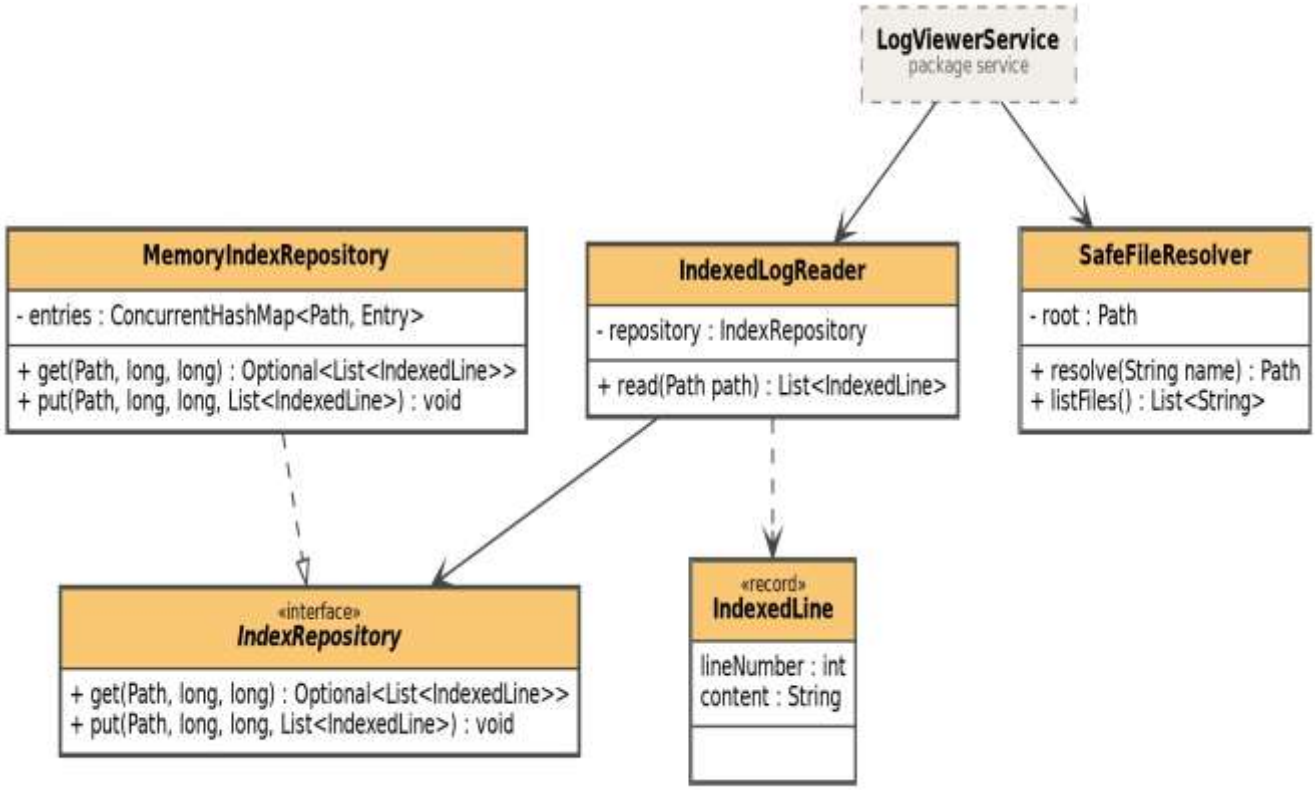
Entry point, UI and service layer

Main builds every object and connects them, then starts either the command-line interface (CliApp) or the Swing window (GuiApp). Both talk only to LogController, which hands the work to LogViewerService (analysis) and ReportService (reports). AnalysisResult is the record returned from an analysis.



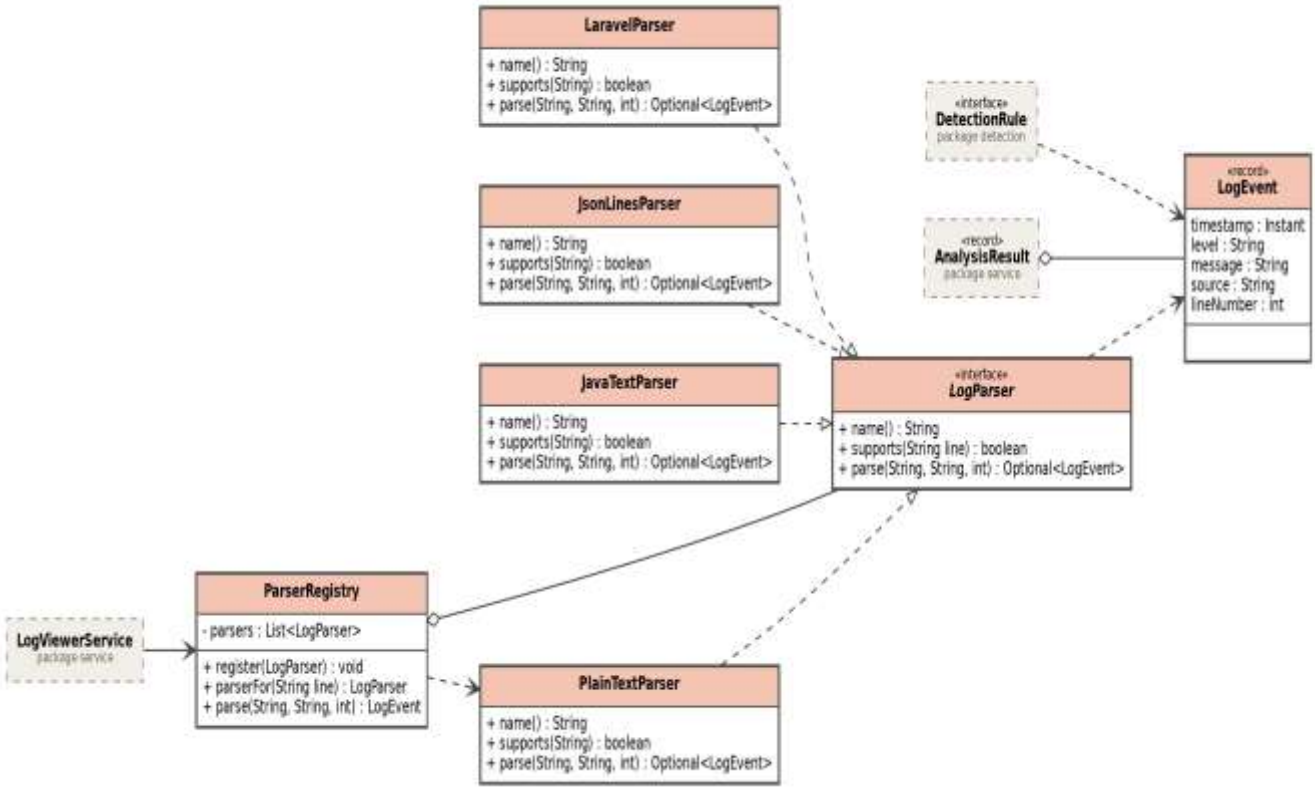
File access and line index

SafeFileResolver turns a user-supplied file name into a real path inside the logs folder and blocks path traversal. IndexedLogReader reads the file into numbered IndexedLine records and caches them through the IndexRepository interface, implemented in memory by MemoryIndexRepository.



Parsing log lines

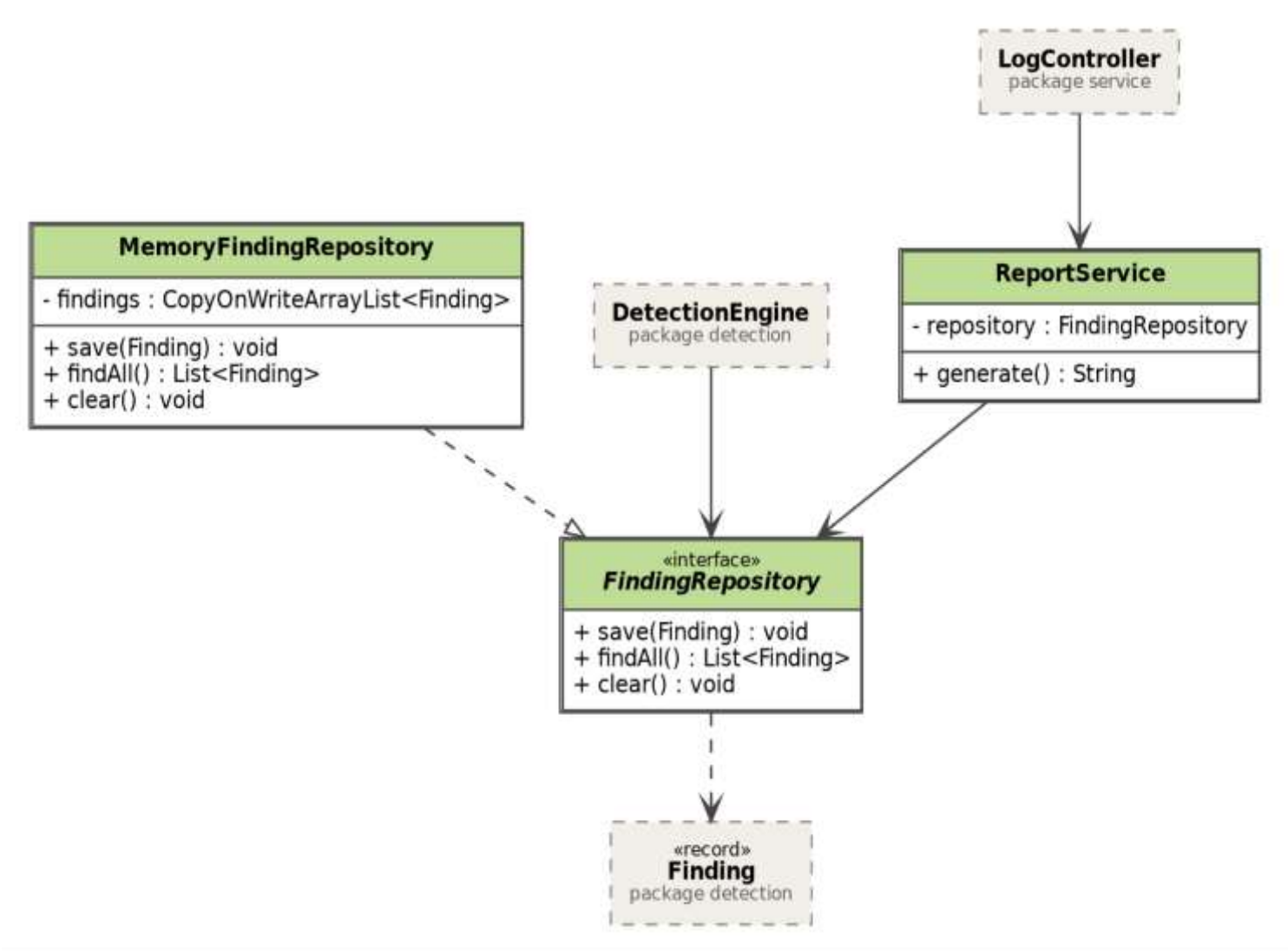
ParserRegistry holds the registered LogParser implementations and tries them in order for every line. The first parser that can read the line produces a LogEvent. PlainTextParser accepts any line and is used as the fallback. A package-private helper, ParserSupport, handles timestamp and level normalisation for all parsers and is not shown.



Detection

DetectionEngine runs every registered DetectionRule against the parsed LogEvents, saves each Finding to the repository and returns them.

RepeatedLoginFailureRule is the one rule currently implemented. For each source IP it checks failed logins against a sliding time window and raises a finding whose severity rises with the number of failures: LOW for the first failed login, MEDIUM for a second within five minutes, and HIGH once the threshold of five failures within five minutes is reached.



Findings storage and reporting

Findings are stored through the FindingRepository interface (in memory via MemoryFindingRepository). ReportService reads all saved findings and formats them into the text report shown by the CLI and GUI.

Key Design Decisions and Trade-Offs

#	Decision	Main alternative	Gained
D1	Layered design with one controller shared by CLI and GUI	UI calling analysis code directly	Both interfaces behave identically; logic is testable without a UI
D2	DetectionRule interface with an engine that runs registered rules	One hard-coded detection method	New rules need no engine changes

D3	Parser registry (Java text, Laravel, JSON Lines, plain-text fallback)	One fixed log format	Handles several formats
D4	Dependency-free, regex-based JSON Lines parsing	A JSON library such as Jackson	Zero dependencies, simple build
D5	Filtered file persistence	A database or files	Simple, fast, swappable
D6	Read whole file and cache it by path, modified time and size	Streaming line by line	Simple; repeat analyses are fast
D7	Sliding-window detection grouped by IP	Fixed time buckets	No missed bursts at bucket edges
D8	Regex heuristics for “failure” and “IP”	Format-specific structured fields	Works across formats
D9	Safe file access by name inside logs/	Free path input	Blocks path traversal and symlink escapes
D10	Java 17 records and manual wiring in Main	Classic classes; a DI framework	Concise immutable data; no framework
D11	SwingWorker and four-worker pool	All work on the UI thread	Responsive UI; shared state needs synchronization

Testing and validation

Test cases and results (current build)

The final build was checked with Junit 5.11.4 and Maven Surefire 3.5.3 Running mvn – Dgui, tests=true verify completed 58 test invocations with zero failures, errors or skipped tests. 57 cores and parameterized checks one Swing workflow test. The build ran on Java 21.0.9 with Java release 17 as the compilation target.

#	Test	Input	Expected result
1	IPv4 and phrases	Boundary addresses, invalid tokens and failure-language cases	Valid boundary Ips accepted; invalid/IPv6 tokens rejected; supported failures detected.

2	Severity and time	Five failures, distinct IPs, out-of-order events and 300/301-second boundaries	LOW, MEDIUM and HIGH milestones; independent counts and correct time-window behaviour
3	Parsing	Supported text/JSON and malformed dates, levels, JSON and nesting	Valid entries parse; malformed entries retained separately; blanks excluded from parsed totals.
4	Files and recursion	Nested files, unsupported extensions, unsafe paths and conflicting imports	Supported paths discovered; traversal and overwrite attempts rejected.
5	Persistenc e	Repeat a file, use../alias and reopen the repository	Identical findings remain unique after reanalysis and reload.
6	Concurren cy	Two tasks held by a latch; one file fails	Two worker threads run; successful results and per-file errors both returned
7	Exports	CSV quotes and formula-like content; saved CSV/text reports	Escaping and file output match expectations; invalid extensions rejected
8	GUI workflow	Analyze all, filter HIGH, search IP, no-match search and analyze clean file	9 findings; HIGH filter 2; IP search 1; no match 0; clean file 0 findings

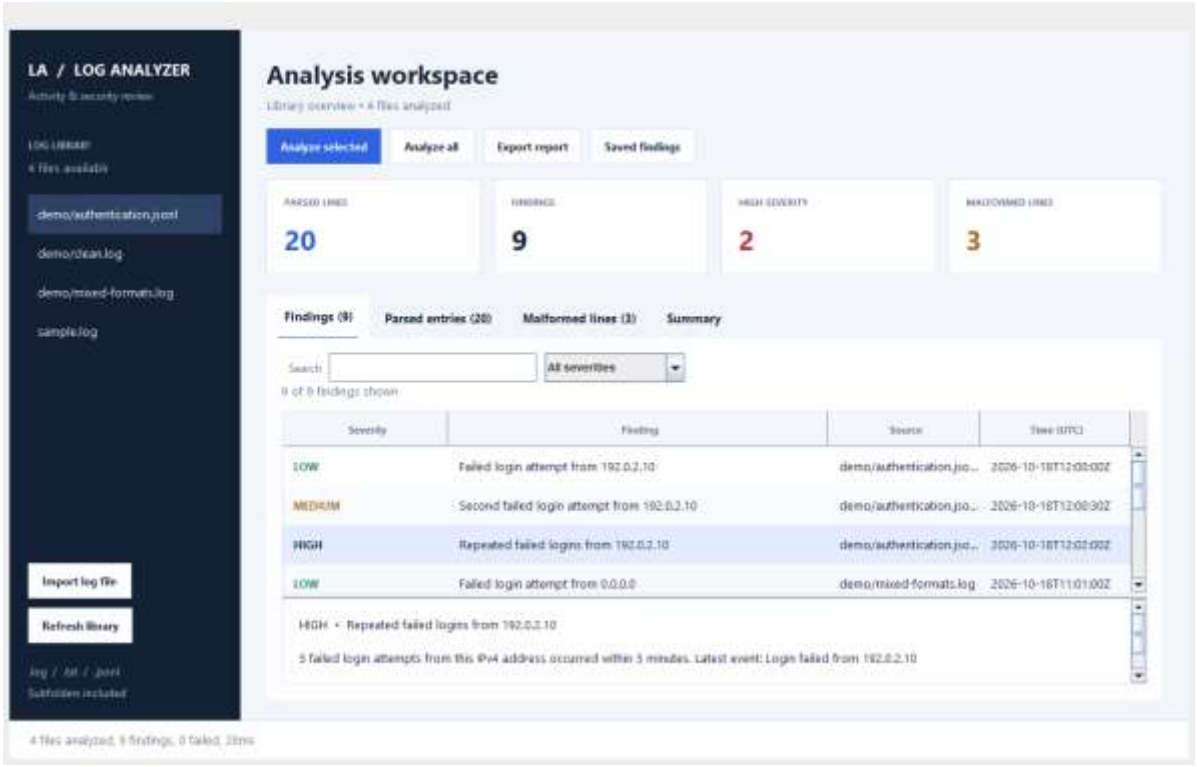


Figure: the final GUI after Analyze all, showing 20 parsed entries, 9 findings, 2 HIGH findings and 3 malformed lines.

The Swing test checks analysis, row selection, severity filtering, text speech, malformed-line display and repeated analysis. It renders the normal and minimum window sizes for layout checking. The test uses the application's components rather than native mouse input. The ordinary mvn verify command skips this opt-in desktop test.

Demonstration output

File	Parsed	Malfromed	Blank	Findings
sample.log	5	0	0	3
demo/authentication.jsonl	6	1	0	3
demo/clean.log	3	0	0	0
demo/mixed-formats.log	6	2	1	3
Total	20	3	1	0

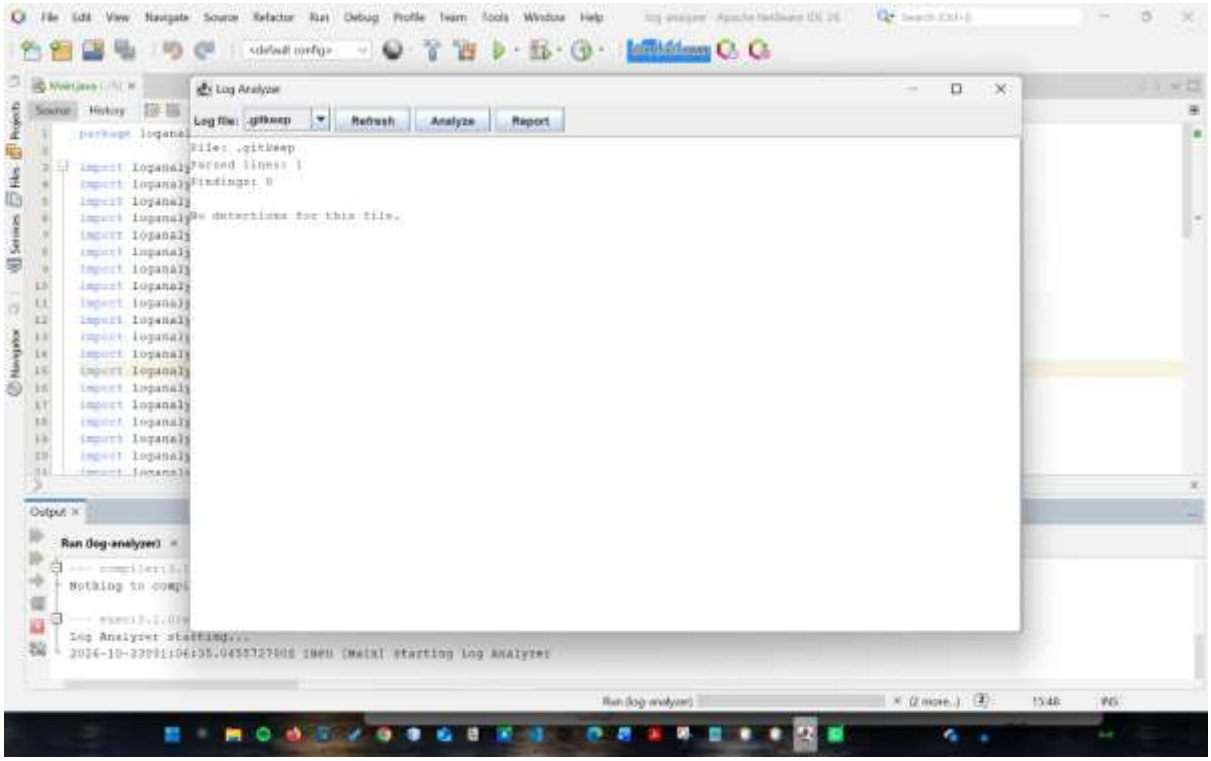
Earlier test runs (before severity levels were added)

The runs below were captured on an earlier build, when every finding was rated HIGH. They are kept as additional evidence that the pipeline works end to end.

The tool analyses log files for host-based intrusion detection. Its primary function is to flag repeated failed logins, which can be a sign of a brute-force attack, by detecting repeated authentication errors within a specific time window.

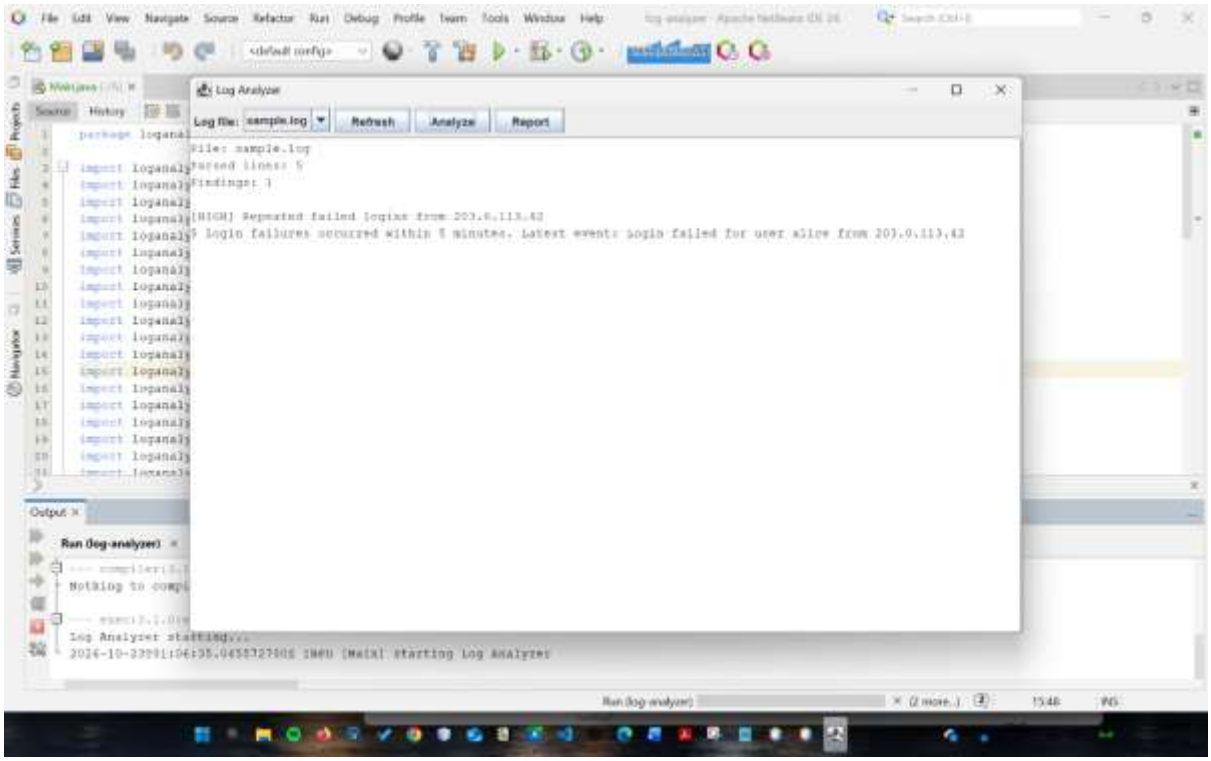
The core analysis logic operates on three main criteria:

- **Identification of a Security Incident:** Looking for failed login attempts (Login failed for user...).
- **Rate Limiting/Correlation Rule:** Evaluating whether a threshold of events occurred within a defined time frame (within 5 minutes).
- **Attacker Profiling:** Extracting the targeted username (alice) and the source IP address (203.0.113.42).



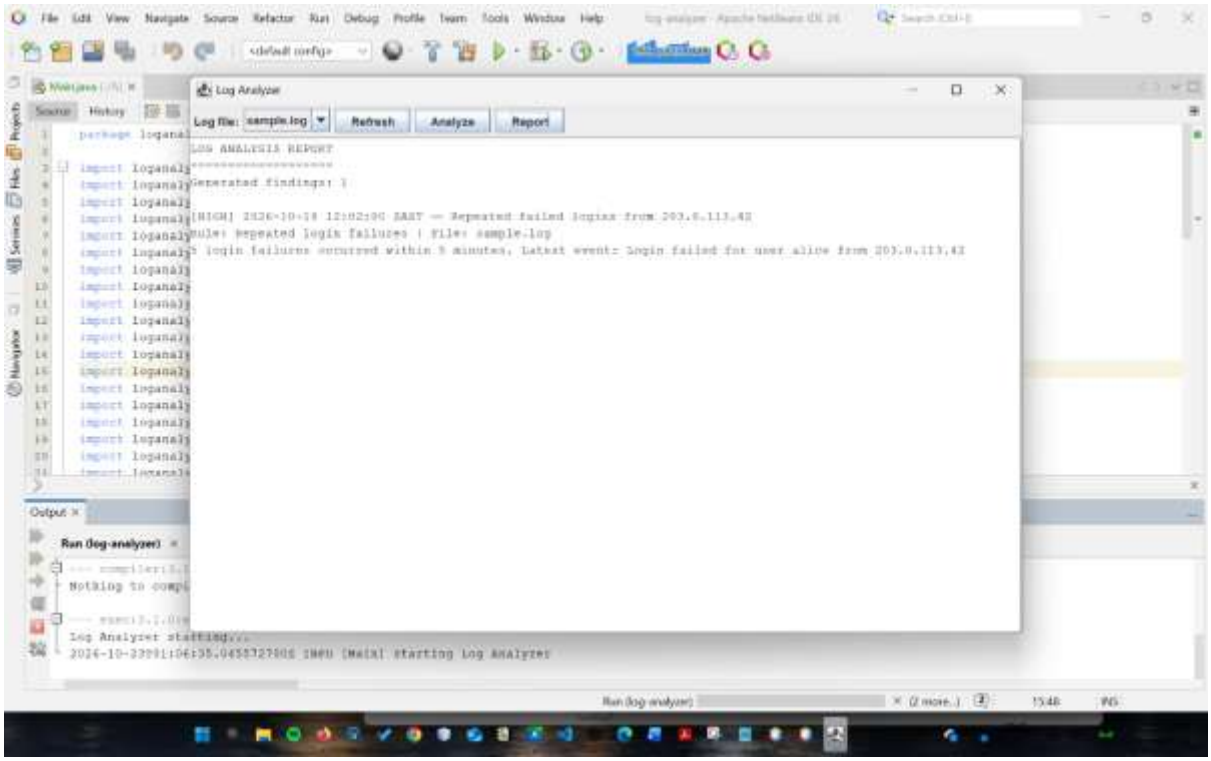
Reported Generation Mode

- **State:** The user has clicked the Report button for the file sample.log
- **Output Analysis:** The text area prints a structured header: LOG ANALYZER.
- **Findings:** It aggregates the detected anomalies and flags them with a severity rating of [HIGH].
- **Security Event Breakdown**:
  - Timestamp: 2026-10-18 12:02:00 SAST.
  - Triggering Rule: Repeated login failures
  - Contextual Description: The tool notes that multiple login failures happened within a 5-minute threshold, concluding with the latest recorded event where user alice was targeted from the external IP address 203.0.113.42 (a standard documentation IP block).
- **Technical Implication:** The pipeline processed 5 log entries, found a cluster of failed logins and produced a finding containing the details of the failures.



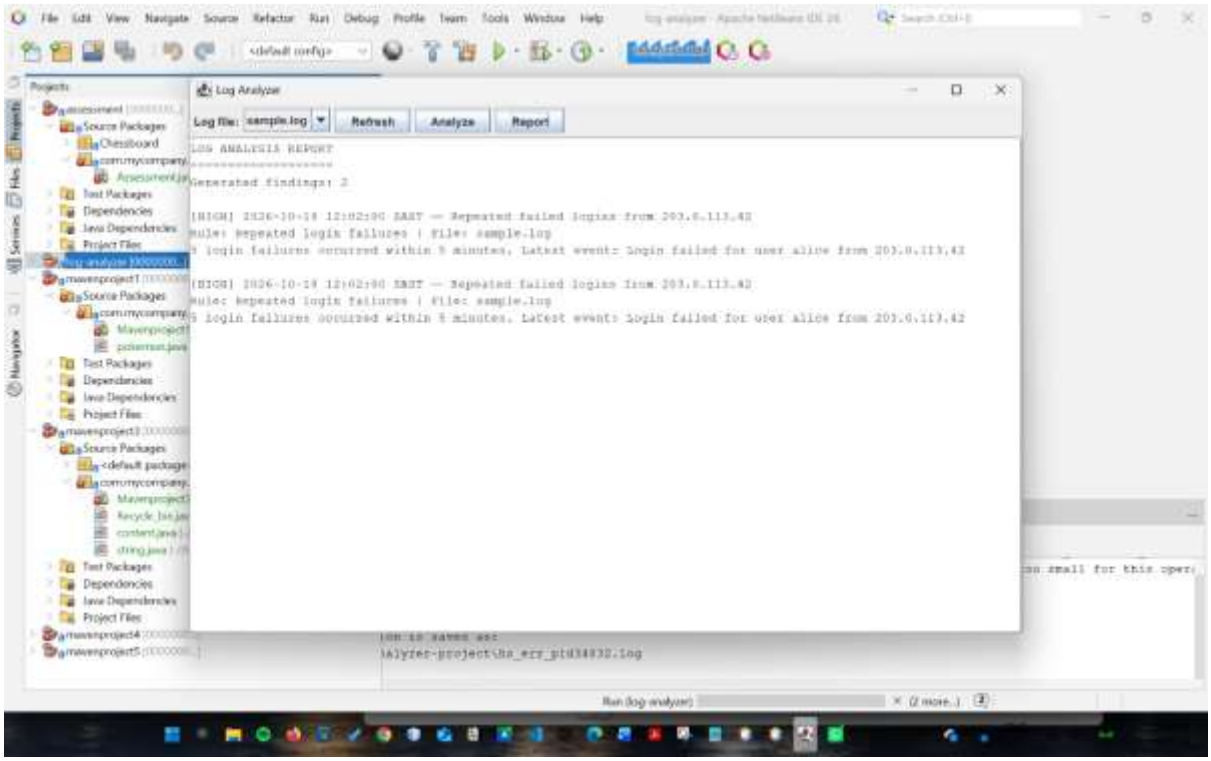
Raw Analysis/Summary Mode

- **State:** The user has clicked the Analyze button for the same file (sample.log).
- **Output Analysis:** Instead of a formalised report header, this mode prints execution metrics and key indicators:
  - Parsed lines: 5 – Shows the file size is tiny, containing exactly 5 lines of log data.
  - Findings: 1 – Confirms that out of those 5 lines, 1 distinct correlated security pattern was matched.
- **Technical Implication:** A correlation engine processed a linear stream of 5 log entries, found a cluster of authentication errors, and generated a single security alert containing the details of the failure.



Baseline/Negative Test Control

- **State:** The analyzer is run against a file named .gitkeep.
- **Output Analysis:**
  - Parsed lines: 1
  - Findings: 0
  - No detections for this file.
- **Technical Implication**: This serves as a vital control test for the software's parsing logic. The program gracefully processes a non-log file or empty placeholder without crashing, correctly registering 0 alerts since no matching security patterns exist.



Final Reading

- **State:** The output is consistent with the pipeline in architecture. The selected file is read, parsed into events, passed to the detection engine and evaluated by the rule.
- **Output Analysis:**

Field	Value	Interpretation
Severity	HIGH	The rule judged the pattern serious
Timestamp	2026-10-18 12:02:00 SAST	Time of the latest triggering event; zone handling (UTC+2) is working
Title	Repeated failed logins from 203.0.113.42	Events are grouped by source IP
Rule	Repeated login failures	RepeatedLoginFailures fired
Source	Sample.log	File name is traced into the finding
Description	5 failures within 5 minutes; latest event: user alice	Threshold and window conditions were met

Correct behaviour: the detection logic worked as designed. Five failed logins from one IP inside a five-minute window were identified and rated HIGH.
Anomaly: when Analyze was run more than once on the same file, the report listed findings that were identical in every field, including the latest-event

timestamp. This happens because the findings repository is never cleared between runs (see Limitations).

- **Technical Implications:**

Architecture:

- The pipeline (read, parse, detect, format, display) is functioning end to end, and the Finding fields map directly onto the report layout. This supports the architecture described in this report.

User guide (README)

Overview

The Log Analyzer reads authentication log files, turns each line into a structured event, applies the detection rule and reports suspicious login activity with a severity level.

Requirements and setup

- Java 17 and Apache NetBeans.
- Place the log files to analyze in the logs/ folder. The application opens files only from this folder and rejects paths that try to leave it.

Running the application (Apache NetBeans)

1. Open the project in NetBeans.

2. Right-click the project, choose Properties, then choose Run.

3. Set Main Class to loganalyzer.Main.

4. Set Arguments to --gui (two ordinary hyphens) to start the GUI, or leave Arguments blank to start the command-line interface.

5. Click OK, then run the project with F6.

Command-line interface

- list: shows the log files in the logs/ folder.
- analyze <file>: runs the detection engine on a file, for example analyze sample.log.
- findings: shows the security findings produced so far.
- report: prints the full text report.
- help: shows the available commands.
- exit: closes the application.

Graphical interface

1. Choose a log file from the drop-down list, for example sample.log. Click Refresh if you have added files to logs/.

2. Click Analyze. The Analysis Summary shows the parsed lines, malformed lines and number of findings, and the Results list each finding with its severity (LOW, MEDIUM or HIGH).

3. Read the status bar at the bottom of the window, for example sample.log: 5 lines, 3 findings, 0 malformed, 69ms.

4. Export report saves CSV or text for the full current analysis. Saved findings shows historical results.

Known limitation: the malformed-lines figure currently always shows 0 (see Challenges faced).

Graduate attribute reflections

GA2: Application of scientific and engineering knowledge

- Object-oriented design: the DetectionRule, LogParser, IndexRepository and FindingRepository interfaces and the Strategy pattern let new rules and parsers be added without changing DetectionEngine.
- Data structures: Java collections hold log lines, events and findings in memory, and each IndexedLine keeps its line number.
- Algorithm: for each source IP, failed logins are checked against a sliding five-minute window, and severity rises with the count (LOW, MEDIUM, HIGH). The current version re-scans events for each window, which is quadratic; a linear version is planned.
- File handling: SafeFileResolver restricts access to the logs folder and blocks path traversal, and IndexedLogReader reads each file into numbered lines.
- Regular expressions: precompiled patterns parse lines in the Java text, Laravel, JSON Lines and plain-text formats.

GA7: Sustainability and impact

- Impact: by flagging repeated failed logins, which can be a sign of a brute-force attack, the tool helps administrators notice suspicious activity early, which can reduce downtime and data loss.
- Efficiency: the application uses no external libraries, precompiles its regular expressions and keeps data in memory. The status bar reports the analysis time for each run.
- Trade-off: each file is read fully into memory and cached (decision D6). Repeat analyses are fast, but very large files use more memory. Streaming line by line is a planned improvement that would reduce memory use.

- Resilience: running the analyzer on a non-log placeholder file did not crash it and returned zero findings.

Challenges faced

Technical

- Duplicate findings: findings accumulate in memory, so analyzing the same file again produces duplicates.
- Regex edge cases: a username such as "bad" caused a false finding, and an embedded time such as 12:30:45 caused a missed attack. A slow attack of 10 events produced 6 findings, which is noisy.
- Malformed-line counting: this was written as a standalone prototype that counts non-blank lines failing a fixed format pattern, but it was not connected to the parsing pipeline before submission. The summary therefore always shows 0 malformed lines, even for files with broken lines. Connecting the counter to the parsers is a planned fix.
- Circular dependency between the detection and reporting layers, which is planned to be refactored.
- No automated unit tests yet; testing was done manually.

Teamwork

Attendance at some recent team meetings was uneven, which made it harder to know which features were finished before integration. The project lead coordinated the module merges, final verification and build packaging, and took on some testing and documentation checks during integration.

Team contribution breakdown

Roles as assigned in the group plan:

Member	Role	Responsibilities
Milton	Project lead and integration	Timeline and Gantt chart, Git branches and merges, final integration, final build and zip; also took on some testing and documentation checks
Reece	Log parsing and file handling	SafeFileResolver, LogFile, ParserRegistry, the LogParser interface and parsers producing LogEvent objects; malformed-line counting
Reginald	Detection rules	DetectionRule interface and RepeatedLoginFailureRule; IP regex restriction, a second regex and severity levels (with Molly)
Molly	Detection engine and alerts	DetectionEngine and Finding objects; IP regex restriction, a second regex and severity levels (with Reginald)
Iviwe	Findings, reporting and logging	FindingRepository, MemoryFindingRepository, ReportService and the report output; application logging; possible export and summary
Jamaine	User interfaces	Command-line interface and Swing GUI, GUI polish and status bar; Gantt chart (with Milton)
Tchitembo	Testing and sample data	Sample log files, manual test cases and the test table; JUnit tests (planned)
Somila	Documentation	Problem statement, README and user guide, GA2 and GA7 reflections, challenges, conclusion and report finalisation

Limitations and future improvements

Limitations

Detection

- Only one rule (repeated failed logins), grouped by IP only.
- Severity levels are fixed: LOW for the first failed login, MEDIUM for a second within five minutes and HIGH at five within five minutes. The threshold (5) and window (5 minutes) are hard-coded.
- Slow attacks produce noisy repeats (verified: 10 events gave 6 findings).

Accuracy

- The IP regex is too loose (verified): a username bad caused false finding, and an embedded time like 12:30:45 caused a missed attack.
- No syslog/auth.log parser (verified): such lines get a 1970 timestamp and are silently ignored.

Performance

- The detection algorithm is quadratic (verified).
- Findings accumulate across Analyze clicks because the repository is never cleared (verified).
- No automated tests exist.
- Detection and reporting depend on each other (circular)
- No persistence, no export, and findings lack line numbers.
- The dropdown lists .gitkeep even though only log files are intended.

Future improvements

Horizon	**Improvements**
Short-term	Linear sliding-window algorithm, SwingWorker, line numbers in findings, break the circular dependency, configurable settings
Medium-term	More rules (password spraying, success after failures, off-hours login), syslog parser, line-by-line streaming, CSV/HTML export, SQLite storage
Long-term	Live tailing, email or webhook alerts, IP reputation, anomaly detection, dashboard

Conclusion

The Log Analyzer meets its main objective. It reads local log files in several formats, converts them into structured events, and flags repeated failed logins from the same IP address with LOW, MEDIUM and HIGH severity, through both a command-line interface and a Swing GUI. The layered design, with a DetectionRule interface and one controller shared by both interfaces, makes new rules easy to add and keeps the two interfaces consistent.

Several known issues remain: duplicate findings on repeated analysis, and a loose IP pattern. These are the priorities for the next version, together with report export, streaming of large files and further detection rules.

References

Heiberg, T. (2021) 'Transnet restores operations at ports after cyber attack', CNBC Africa, 29 July. Available at: https://www.cnbcafrica.com/2021/transnet-restores-operations-at-ports-after-cyber-attack/ (Accessed: 1 November 2026).

Shabalala, Z. and Heiberg, T. (2021) 'South Africa's Transnet hit by cyber attack affecting some container terminal operations - sources', CNBC Africa, 22 July. Available at: https://cnbcafrica.com/2021/south-africas-transnet-hit-by-cyber-attack-affecting-some-container-terminal-operations-sources (Accessed: 1 November 2026).